

Chapter 4

CS1 course package - University of Southern Indiana (USI)

Srishti Srivastava

University of Southern Indiana

fsrishti@usi.edu

Abstract

The growing importance of parallel and distributed computing (PDC) in modern computing systems calls for early exposure to PDC concepts in undergraduate computer science education. Yet, introductory programming courses such as CS1 have traditionally focused on sequential programming paradigms, leaving students under prepared for the realities of contemporary computing environments. This chapter presents a course exemplar developed at the University of Southern Indiana (USI), in which a CS1 course titled, “Introduction to Object-Oriented Programming”, was redesigned to include PDC concepts throughout a semester via experiential and active learning pedagogy. This chapter describes the motivations, design decisions, and structural changes that guided the course redesign, detailing how PDC principles were incrementally and meaningfully integrated alongside core object-oriented programming concepts. Through carefully crafted hands-on activities, collaborative exercises, and project-based learning experiences, students are introduced to foundational PDC ideas in a contextualized and accessible manner from the very beginning of their computing education. The chapter further shares insights drawn from the implementation of this redesigned course, discussing observed impacts on student engagement, conceptual understanding, and programming competency. Practical recommendations are offered for instructors seeking to undertake similar curriculum transformations in their own CS1 courses, with the aim of equipping the next generation of programmers with the parallel and distributed computing mindset that modern software development demands.

Descriptor Elements

Programming Language Employed: Java

Textbook Used: Starting Out With Java: Control Structures through Objects, 8th edition, Pearson Revel 2022, by Tony Gaddis.

Relevant core courses: CS258 (CS1): Introduction to Object Oriented Programming

Pre-requisites: At USI, students enrolling in CS1 are required to have completed or be concurrently enrolled in linear algebra as a prerequisite or co-requisite. In practice, the majority of incoming students satisfy this requirement through placement scores prior to beginning the program.

Relevant PDC topics: Table 4.1 lists the integrated PDC topics with bloom’s classification (Remembering (RE), Understanding (UN), Applying (AP), Analyzing (AN), Evaluating (EV), and Creating (CR)).

Table 4.1: USI CS1 Relevant PDC Topics and Blooms Levels

PDC Concept	Chapter Section				
	Unplugged Flag Maker Activity (4.3.1)	Unplugged Penny Activity (4.3.2)	Plugged-In Flag Maker Activity (4.4.1)	Plugged-In Parallel Sort Penny Activity (4.4.2)	Plugged-In Greenfoot Activity (4.4.3)
Data/Task Parallelism	RE, UN, AN	RE, UN, AN	RE	RE, UN, AP, AN, EV	RE
Shared Memory	RE, UN	RE, UN	RE	RE, UN	RE
Scheduling/Load Balancing	RE, UN, AN	RE, UN, AN	RE	RE	RE
Speedup/Amdahl's Law	RE, UN, AN	RE, UN, AN	RE	RE, UN, AN, EV	RE
Event Handling					RE, UN

Learning outcomes from PDC interventions:

1. FlagMaker Activity : Data Parallelism (Plugged-In& Unplugged Activity)
 - (a) Define the term data parallelism as it applies to distributing a computational workload across multiple processors.
 - (b) Distinguish between sequential computing and parallel computing using examples from the flag-drawing task.
 - (c) Explain why parallel execution time is bounded by the slowest processor rather than the average processor time.
 - (d) Recall the definitions of key PDC vocabulary including multicore, multiprocessor, synchronization, and scheduler.
 - (e) Explain what contention means when multiple processors compete for a shared resource.
 - (f) Compare the completion times recorded across the single-processor, two-processor, and four-processor scenarios of the unplugged activity.
 - (g) Apply loop-based indexing to correctly partition a flag grid into horizontal stripes of specified heights.
 - (h) Use conditional logic within a loop to assign the correct color to each cell based on its row position.
 - (i) Differentiate between a row-based data partition and a column-based data partition in terms of data access patterns.
2. Penny Sorting Activity : Data/Task Parallelism, Speedup, and Amdahl's Law (Unplugged & Plugged-In Activity)
 - (a) Define speedup as the ratio of sequential execution time to parallel execution time for the same task.

- (b) Describe what load imbalance means when one processor receives significantly more work than others.
 - (c) Explain scalability as a parallel system's ability to increase performance proportionally as more processors are added.
 - (d) Identify which phase of the penny sorting activity models a load-imbalanced parallel execution.
 - (e) Compute speedup values from timing data collected during the sequential and parallel phases of the penny sorting activity.
 - (f) Implement a sequential sort in Java using a selection sort, insertion sort, or bubble sort algorithm to order an array of integer penny years.
 - (g) Use *Arrays.parallelSort* in Java to perform a parallel sort on the same array of penny years.
 - (h) Differentiate between data parallelism and task parallelism by identifying which model applies to each phase of the penny sorting activity.
 - (i) Examine how the unequal distribution of pennies in Phase 3 of the unplugged activity increases overall parallel execution time despite involving more processors.
 - (j) Contrast the theoretical speedup predicted by Amdahl's Law with the observed speedup from the recorded timing data.
 - (k) Explain why adding more processors does not always result in proportionally faster execution.
3. Greenfoot Plugged-In Activity : A Sneak Peak into Event Handling
- (a) Identify event-driven programming as a paradigm in which program execution is determined by user-generated or system-generated events.
 - (b) Describe how an event listener responds to a specific event such as a key press or mouse click in a Greenfoot scenario.
 - (c) Recognize the role of actors and the world class in structuring a Greenfoot program.

Context for use: The PDC modules described in this chapter were implemented in CS 258, Introduction to Object-Oriented Programming, a CS1 course at the University of Southern Indiana (USI), a public master's-level institution located in Evansville, Indiana. USI enrolls approximately 9,750 students across more than 130 areas of study and holds Carnegie classifications as a community-engaged institution. The Computer Science program resides within the College of Business and comprises seven full-time faculty members. CS 258 is a mandatory CS1 course required of all Computer Science and Computer Information Science majors as well as CS minors. The programming language used for the course is Java. The course introduces the fundamental concepts of programming from an object-oriented perspective, covering data abstraction, inheritance, overriding, programming flow of control, operator precedence, and introductory data structures such as lists and arrays. It also addresses human-computer interfaces, graphics, and the social implications of computing. In a semester, the course is taught over 16 weeks with 50-minute or 75-minute sessions held thrice or twice per week. The

class enrolls approximately 35–40 students on average. Students are primarily at the freshman level, with prior programming experience ranging from none to introductory. Table 4.4 shows the classification of students taking the CS1 course across Fall 2024 (baseline semester) and Fall 2025 (PDC intervention semester). Table 4.7 shows incoming programming experience of students entering CS1. The class included a mix of Computer Science majors, Computer Information Science majors, and Mathematics majors completing a double major with Computer Science. The student distribution by majors is shown in Table 4.5. Almost eighty percent of the students in CS1 identify as male and approximately 13.4% identify as female as shown in Table 4.2 and student race is shown in Table 4.3. Table 4.6 shows incoming student experience of general computing such as internet searches, email, discord, discussion groups, games, word processing, spreadsheets, organizing files, and organizing folders. With respect to prior knowledge of PDC, no baseline PDC instruction had been provided to students before the modules described here.

Table 4.2: USI CS1 Gender Distribution

Gender	Fall 2024 (n=26)	Fall 2025 (n=41)	Total (n=67)
Male	24	29	53 (79.1%)
Female	2	7	9 (13.4%)
Other	0	1	1 (1.5%)
No Response	0	4	4 (6.0%)

Table 4.3: USI CS1 Race/Ethnicity Distribution

Race/Ethnicity	Fall 2024 (n=26)	Fall 2025 (n=41)	Total (n=67)
White / Caucasian	20	30	50 (74.6%)
Black / African American	3	4	7 (10.4%)
Hispanic	1	3	4 (6.0%)
Asian / Pacific Islander	2	1	3 (4.5%)
Two or more ethnicities	0	1	1 (1.5%)
No Response	0	2	2 (3.0%)

Table 4.4: USI CS1 Class Standing

Class Standing	Fall 2024 (n=26)	Fall 2025 (n=41)	Total (n=67)
First year (freshman)	11	14	25 (37.3%)
Sophomore	8	13	21 (31.3%)
Junior	7	13	20 (29.9%)
Senior	0	1	1 (1.5%)

Table 4.5: USI CS1 Declared Majors

Major	Fall 2024 (n=26)	Fall 2025 (n=41)	Total (n=67)
Computer Science	22	40	62 (92.5%)
Computer Information Systems	3	1	4 (6.0%)
Computer Engineering	0	1	1 (1.5%)
Math	1	0	1 (1.5%)
Other (biology)	1	0	1 (1.5%)

Students could select multiple majors; percentages are of students and therefore sum to more than 100%.

Table 4.6: USI CS1 Self-Reported General Computing Experience

Computing Experience	Fall 2024 (n=26)	Fall 2025 (n=41)	Total (n=67)
No experience at all	1	0	1 (1.5%)
Very little experience	1	0	1 (1.5%)
Slightly experienced	0	3	3 (4.5%)
Between slightly and moderately experienced	3	5	8 (11.9%)
Moderately experienced	5	6	11 (16.4%)
Very experienced	12	11	23 (34.3%)
Extremely experienced	4	16	20 (29.9%)

Table 4.7: USI CS1 Self-reported prior programming experience of CS1 participants

Programming Experience	Fall 2024 (n=26)	Fall 2025 (n=41)	Total (n=67)
No experience at all	2	4	6 (9.0%)
Very little experience	4	11	15 (22.4%)
Slightly experienced	7	11	18 (26.9%)
Between slightly and moderately experienced	2	5	7 (10.4%)
Moderately experienced	9	7	16 (23.9%)
Very experienced	1	3	4 (6.0%)
Extremely experienced	1	0	1 (1.5%)

4.1 Introduction

Many instructors teaching CS1 courses face a genuine and understandable challenge that the course already carries a substantial conceptual load, covering object-oriented design, control flow, data structures, and software engineering fundamentals, and the prospect of adding PDC content without displacing core learning objectives can feel daunting. As a result, PDC integration in CS1 continues to be the exception rather than the norm. This chapter is written for instructors who share that concern but are looking for practical, tested strategies to resolve it. Specifically, it is written for instructors at institutions similar to USI, a mid-sized public master's university with a community-engaged mission, where resources are not unlimited, class sizes are modest, and students arrive at CS1 with widely varying levels of prior programming experience. The strategies described here do not require additional instructional time beyond a standard 50-minute to 75-minute class session, do not presuppose access to parallel computing hardware or specialized software environments, and are designed to complement rather than compete with the existing learning objectives of an introductory object-oriented programming course.

The approach taken in this work is grounded in active learning and the use of unplugged activities as bridges to computational thinking. Unplugged activities, in which students physically enact computational processes without the use of a computer, have a well-established history in CS education as tools for making abstract concepts tangible and accessible. Their effectiveness is particularly well-documented at the introductory level, where students are simultaneously learning to program and to think algorithmically, and where the cognitive distance between a concept and its implementation can otherwise be prohibitively large [1, 2, 4, 5]. By first giving students a physical, embodied experience of parallel execution, and then connecting that experience to a programming task that operationalizes the same concepts, the modules described in this chapter follow a concrete-to-abstract pedagogical sequence that is also consistent with Bloom's taxonomy of cognitive development.

Three PDC modules containing five activities are presented. The first module focuses on data parallelism and uses a flag-drawing activity, first as a loop-based Java programming assignment implementing serial flag coloring, followed by an unplugged paper-and-marker form, to illustrate how a computational task can be partitioned across multiple processors. The second module introduces data and task parallelism, speedup, and Amdahl's Law through a penny sorting and searching activity, again bridging an unplugged experience to a plugged-in implementation using Java arrays and parallel sorting methods. The third module provides a brief introduction to event-driven programming through a demonstration of Greenfoot, offering students a preview of paradigms they will encounter more formally in their follow-up CS2 course.

4.2 How CS1 was changed and PDC topics integrated

A central design principle of the approach described in this chapter is that PDC concepts can be integrated into a CS1 course without restructuring the syllabus, displacing core content, or requiring additional class sessions. Rather than treating PDC as a standalone topic

that competes for time alongside the traditional CS1 curriculum, the integration strategy adopted here embeds PDC thinking directly into the instructional contexts where foundational programming concepts are already being taught. The result is a course that looks, from a structural standpoint, nearly identical to a conventional CS1 offering, but in which selected modules are taught through a lens that naturally introduces parallel reasoning.

4.2.1 Integrated Syllabi (Pre and Post)

The course calendar from the syllabus for CS1 (Fall 2025) at USI as shown in Table 4.8. It spans 16 weeks across nine content modules, covering the standard CS1 arc from introductory programming and Java fundamentals through decision structures, loops, methods, classes and objects, and arrays. The PDC interventions, highlighted in the course calendar, are threaded into this existing structure at three points, each chosen because the underlying CS1 content provides a natural and mutually reinforcing context for the PDC concept being introduced.

Table 4.8: USI CS1 Fall 2025 Course Syllabus Calender with Highlighted PDC Content

Modules	Week	Assigned Reading	Assignments	Quizzes and Exams
Module 1: Introduction to Computers and Java	Week 1	Syllabus, Start Here Content, Instructor Notes, Textbook Chapter 1 (Pearson Revel)	Chapter 1 programming challenge and Pre-Course Experience Survey	Textbook Quiz 1.3, 1.4, 1.5, 1.6
Module 2: Java Fundamentals (Variables, Data Types, Arithmetic Operators, Comments, Console input output, etc.)	Week 2	Textbook Chapter 2 (Pearson Revel)	Chapter 2 Programming Project 1 and 2	Textbook Quiz 2.1, 2.2, 2.3, 2.4, 2.5, 2.7, 2.8, 2.9, 2.10, 2.13, 2.15
Module 3: Decision Structures	Weeks 3-4	Textbook Chapter 3 (Pearson Revel)	Chapter 3 Programming Project 1 and 2	Textbook Quiz 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.8, 3.9

Module 4: Loops and Files	Weeks 5-6	Textbook Chapter 4 (Pearson Revel)	Chapter 4 Programming Project 1, 2 and 3 AND Unplugged Flag Maker Activity (4.3.1) and Flag Maker Plugged-In Activity (4.4.1) as midterm assignment)	Textbook Quiz 4.1, 4.2, 4.4, 4.5, 4.6, 4.7, 4.9, 4.10
Module 5: Methods	Weeks 7-8	Textbook Chapter 5 (Pearson Revel)	Chapter 5 Programming Project 1 and 2	Textbook Quiz 5.1, 5.2, 5.4
Module 6: A First Look at Classes	Weeks 9-10	Textbook Chapter 6 (Pearson Revel)	Chapter 6 Programming Project 1 and 2	Textbook Quiz 6.1, 6.2, 6.3, 6.4
Module 7: Ar- rays and Ar- rayList Class	Weeks 11-12	Textbook Chapter 7 (Pearson Revel)	Chapter 7 Programming Project 1 and 2	Textbook Quiz 7.1, 7.3, 7.4 , 7.8, 7.9, 7.10, 7.11, 7.13
Module 8: A Second Look at Classes and Ob- jects	Weeks 13-14	Textbook Chapter 8 (Pearson Revel)	Chapter 8 Programming Project 1, 2, and 3	Textbook Quiz 8.1, 8.3, 8.4, 8.5
Module 9: Final Project Work Weeks/ Lab Days	Weeks 15-16	Greenfoot Plugged-In Demo (4.4.3) (Event Driven programming)	Unplugged Penny Sorting Exercise (4.3.2) and Parallel Penny Sort Plugged-In Activity (4.4.2) Due as Final Project	Post-Course Experience Survey

4.2.2 Highlights

The vast majority of the CS1 course structure remained unchanged. All nine content modules were retained in their original form and sequence, covering the same textbook chapters, the same auto-graded programming projects, and the same quiz schedule. The course continued to use the Pearson Revel platform for readings and textbook-aligned assessments, and no existing quiz, exam, or core programming assignment was removed from the calendar to make room for PDC content.

In Week 6, one of the standard Chapter 4 programming assignments was replaced by the Flag Maker Plugged-In Activity. This substitution was the only deletion made to the existing assignment structure. This activity was designed to assess the same loop-based programming skills that the displaced assignment would have evaluated, so the change carried no loss in coverage of the Chapter 4 learning objectives. It did, however, add a new in-class component in the form of the Unplugged Flag Maker Activity, which was conducted during a regularly-scheduled class session in Week 6 without extending beyond the allotted 75 minutes.

In Weeks 15 and 16, the course’s existing final project slot was repurposed. Rather than assigning a generic open-ended or object-oriented capstone project as might be conventional in a CS1 course, the Parallel Penny Sort Plugged-In Activity was designated as the final project deliverable. The project submission deadline and the final exam day remained unchanged. The Greenfoot Plugged-In Demo, delivered in the same period, replaced what would otherwise have been an unstructured lab or review session during the final project work weeks, requiring no displacement of any assessed content. The pre- and post-course experience surveys, administered in Weeks 1 and 16 respectively, were added as brief in-class activities and required approximately ten minutes of class time each, well within the buffer that naturally exists at the start and end of a semester.

4.2.3 Challenges

The challenges encountered during PDC intervention are described as: challenges of adoption, challenges of adaptation, and challenges arising from the research layered on top of instruction.

Adoption challenges: the most immediate obstacle was the in-class coordination of the unplugged activities. Both the Unplugged Flag Maker Activity and Unplugged Penny Sorting Exercise require physical materials, group formation, and timed phases that must be orchestrated within a fixed class period. In our sections of 28–34 students meeting in a standard lecture room, distributing materials, forming groups of five to six, and running three timed phases of Penny Sorting while reserving time for a substantive post activity discussion required careful minute by minute planning in advance. Instructors adopting these activities should budget preparation time accordingly and should not assume that transitions between phases will absorb themselves into the period.

Adaptation challenges: The activities described here are calibrated to a particular room, roster size, and staffing level. Larger sections lengthen the group formation step disproportionately, and rooms with fixed furniture may make the implementation more time-consuming than anticipated. Staffing is the other adjustable variable: having a teaching assistant present to distribute penny bags, manage timing, and circulate between groups during Phases 2 and 3 of Penny Sorting is strongly advisable, and we regard it as close to necessary in sections larger than 30 students. Instructors working under constraints unlike ours should expect to modify phase timings rather than the phase structure itself.

Research challenges: Administering the pre- and post-activity assessments in a way that preserved their research validity without disrupting the flow of the session proved to be a distinct challenge from teaching the activities themselves. The Flag Maker pre-test had to be completed before any PDC content was introduced, which constrained the sequencing of the class period and required explaining to students why they were being asked about

material they had not yet encountered. Some students found this ambiguity uncomfortable, and a few initially misread the pre-test as a graded quiz. Instructors should state explicitly, both verbally and in writing, that pre-test responses carry no grade consequence and that incorrect answers are expected and informative. Moreover, instructors intending to collect data for publication should leave sufficient buffer time before the semester begins to secure IRB approval at their own institutions.

4.3 New Unplugged Activities

4.3.1 Unplugged Flag Maker Activity

Problem Description, Prerequisites, and Learning Outcomes

The Unplugged Flag Maker Activity introduces students to PDC concepts through collaborative paper-based simulations. By progressively moving from sequential execution to parallel processing and resource contention scenarios, the activity helps students build intuition about speedup, scalability, load balancing, and synchronization while promoting active learning and student engagement. The activity is designed for CS1 level students and requires minimal technical prerequisites. Students should have:

- Basic familiarity with how programs execute sequentially.
- An introductory understanding of processes and computing systems.

The **activity materials** are intentionally simple and include:

- Gridded paper to serve as flag sheets
- Markers or crayons (colors used in this activity: red, blue, yellow, and green)
- Student participants assigned roles as processors and a CPU scheduler (usually the instructor), and timer

As for the **learning outcomes**, by the end of the module, students should be able to:

1. Explain Von Neumann sequential execution and CPU scheduling.
2. Describe multi-core processing, concurrency, and data parallelism.
3. Understand and apply data decomposition and load balancing strategies.
4. Calculate and interpret speedup and discuss scalability.
5. Recognize the importance of communication and coordination among parallel processes.
6. Identify and analyze resource contention, file contention, and mutual exclusion.
7. Evaluate trade-offs between increased parallelism and system performance.

Implementation Details

The material used for implementing this activity can be found at this [GitHub link](#). The activity consists of four sequential activities that simulate increasingly complex computing environments:

- Phase I: Sequential Execution: One student acts as a process and another as a scheduler. Students simulate single-processor execution by coloring all cells in a grid. Introduces sequential execution and CPU scheduling.
- Phase II: Dual-Processor Parallel Execution: Two students work in parallel while a scheduler coordinates tasks. Students divide the grid into sections, illustrating data decomposition and parallel execution. Introduces concurrency, load balancing, and speedup.
- Phase III: Four-Processor Parallel Execution: Four students execute tasks simultaneously. Students observe the benefits and limitations of scaling parallel resources. Emphasizes speedup, scalability, and coordination overhead.
- Phase IV: Four Processors with Resource Contention: Students share resources while following strict color-order constraints. A scheduler manages contention using a FIFO approach. Demonstrates file contention, mutual exclusion, and resource sharing challenges.

Table 4.9 summarizes the approximate timing of each component of the activity implemented during a 75-minute class session. The class opened with a brief framing of the day's goals and a pre-activity quiz to establish a baseline measure of students' prior knowledge, followed by a short conceptual introduction to PDC and the vocabulary central to the activity. The bulk of the period was devoted to the unplugged activity itself, in which student groups executed four decomposition scenarios of increasing parallelism, recorded the completion time of each scenario. The session closed with a post-activity quiz and an engagement survey. For more details, please refer to Chapter 1, Section 1.3.1.

Table 4.9: USI CS1 Approximate Timing of Flag Maker Activity

Activity	Duration
Welcome and learning goals	3 min
Step 1: Pre-Test	8 min
Introduction: What is parallel computing? “Digging Holes” video	5 min
PDC glossary: data/task parallelism, sequential vs. parallel computing, multicore, multiprocessor, synchronization, scheduler, race condition	8 min
Step 2 setup: Activity overview, coloring rules (correct vs. incorrect), distribution of flag sheets and markers, group formation	5 min
Scenario 1: Single processor (64 cells; one colorer plus timer); record time	7 min
Scenario 2: Two processors (~4 min run) and discussion (~3 min)	7 min
Scenario 3: Four processors by stripe (~3 min run) and discussion (~3 min)	6 min
Scenario 4: Four processors by column (~3 min run) and discussion (~4 min)	7 min
Whole-class debrief: comparison of recorded times, speedup, load balancing, decomposition strategy	6 min
Step 3: Post-Test	7 min
Step 4: Post-activity student engagement survey	5 min
Closing and takeaways	1 min
Total	75 min

Experience and Teaching Tips

The Flag Maker activity provided a positive teaching experience by demonstrating that complex PDC concepts can be effectively introduced in an introductory computer science course through active, unplugged learning. Survey results indicated high levels of student engagement, enjoyment, collaboration, and interest in parallel computing, with many students reporting that the activities helped clarify misconceptions about parallelism and the relationship between processor count and performance. Although some students suggested improvements such as incorporating code examples, increasing activity variety, and refining the classroom setup, the overall experience showed that the activity successfully promoted understanding of key PDC concepts and provided an accessible framework for instructors seeking to integrate PDC topics into CS1 curricula.

Based on student feedback, assessment findings, and lessons learned from implementing the Flag Maker, several improvements could enhance future teaching sessions. First, incorporating visual aids, code snippets, and simplified programming examples alongside the unplugged activities could help students connect conceptual understanding with actual parallel programming practices, particularly for students seeking a deeper technical perspective. Second, rotating student roles (e.g., processor, scheduler, observer) throughout the activities helps increase participation and ensure that all students experience multiple perspectives of parallel systems. Third, providing students with a brief concept guide or terminology hand-out before the activity may improve comprehension and reduce confusion during execution. Additionally, to strengthen reflection and knowledge retention, instructors could incorporate short debriefing discussions or reflection exercises after each activity phase.

Data and Analysis

The data collection in this study was done via (i) single group pre/post test to measure student learning, where each student was graded on six PDC multiple choice questions testing their knowledge on sequential processing, parallel processing, speedup, contention, scalability, and race condition. (ii) A post activity survey was collected from participants to measure student engagement.

Pre/Post Test Analysis:

The pre/post test analysis demonstrates a substantial improvement in student understanding of parallel computing concepts. Among the 24 students who completed both assessments, the average score increased from 3.67 to 5.13 out of a total score of 6, representing a mean gain of 1.46 points. Nineteen students improved their scores, five showed no change, and no students performed worse on the post-test. Statistical analysis confirmed that the improvement was highly significant (Wilcoxon signed-rank test, $p < .001$; paired t-test, $p < .001$), indicating a meaningful learning impact. The greatest improvements occurred on concepts related to multi-processor drawbacks and speedup, while gains were observed across nearly all PDC concepts. Overall, the results suggest that the FlagMaker intervention was highly effective in increasing students' understanding of fundamental PDC concepts and produced substantial learning gains without any decline in student performance. Figure 4.1 illustrates the comparison of the percentage of correct answers for each of the six questions in the pre- and post-tests.

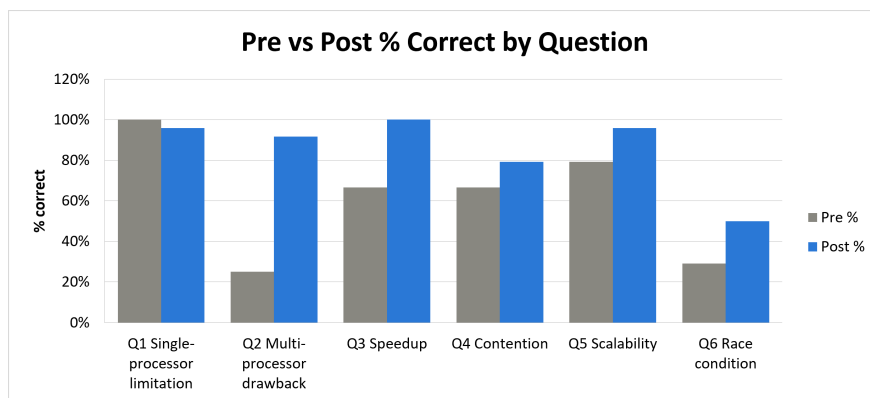


Figure 4.1: USI CS1 Pre vs Post Percentage of Correct Questions for N=24

Post Activity Student Engagement Survey Analysis:

Student engagement with the FlagMaker activity was strongly positive. The survey contained 18 five-point Likert items (Strongly Disagree = 1 to Strongly Agree = 5), two open-text questions (Q2: most interesting thing learned; Q3: suggested improvements), and optional demographics and was filled out by 27 students. Across the 18 Likert items the overall mean was 4.37 out of 5, with 16 items at 85% agreement or higher. Instructor-related items and peer/group items scored highest, and students reported enjoying the activity and seeing it as relevant to current course topics. One item stands clearly apart: “The activity increased my understanding of loops” (mean 3.48, 52% agreement, 33% neutral) was the only item to

draw any disagreement and is the single notable weak spot. The diverging bar chart in Figure 4.2 below shows the distribution of responses for each item, centered on neutral, sorted from highest to lowest mean. Bars extending right indicate agreement; the few extending left indicate disagreement. Item mean is shown at right in the figure.

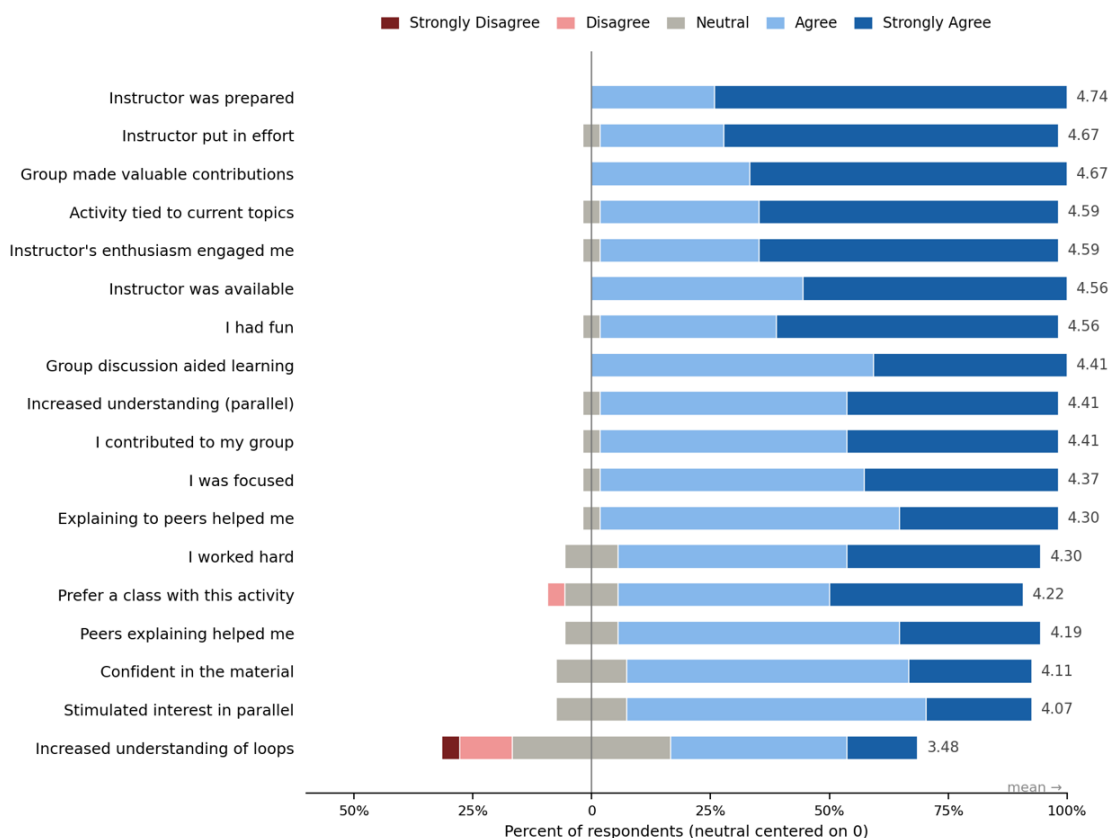


Figure 4.2: USI CS1 Response Distribution by Item

In the open response questions, students most often named contention, core parallelism concepts, and the diminishing returns of adding processors as their most interesting take-aways, which aligns with the concepts that improved most on the paired pre/post test. Suggestions for improvement were dominated by satisfaction (“nothing to change”) or by the improvements in physical logistics of the coloring task (paper size, table space, markers). Responses to the two open questions were coded into the themes as shown in Figure 4.3 along with the theme frequencies.

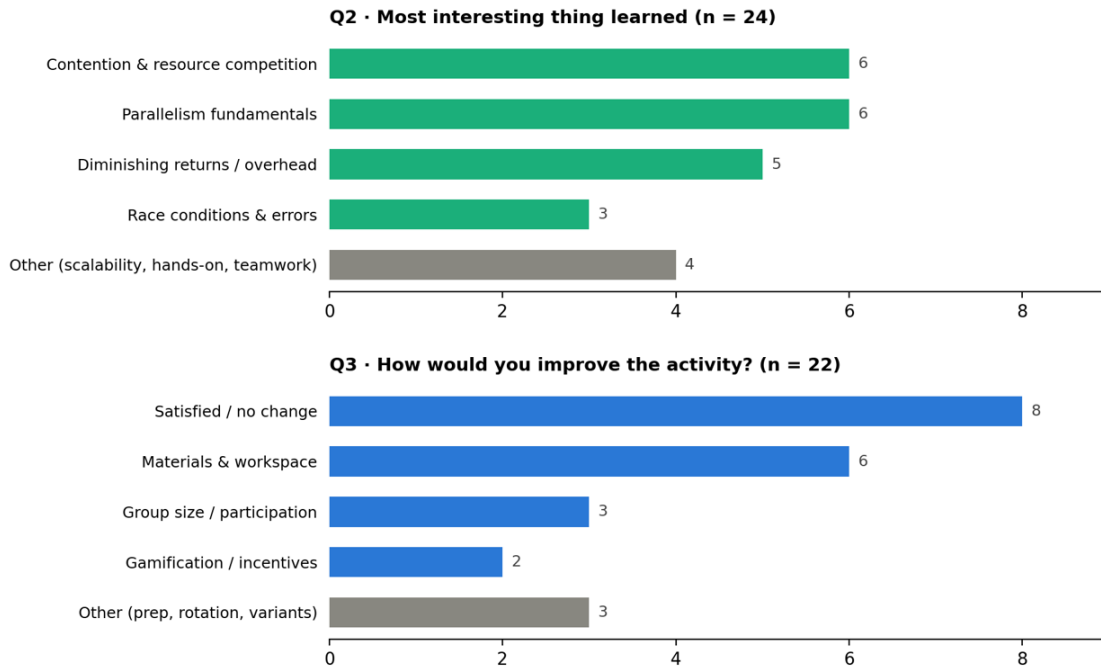


Figure 4.3: USI CS1 Thematic Analysis of Open Response Survey Questions

4.3.2 Unplugged Penny Sorting Exercise

Problem Description, Prerequisites, and Learning Outcomes

The Unplugged Penny Sorting Search Exercise (PE) is a hands-on learning activity designed for CS1/CS2 students, requiring no prior parallel computing background beyond basic algorithmic thinking. The problem description centers on the physical task of sorting and searching through a collection of pennies to identify the oldest coin, using this tangible exercise to model processor roles. The activity’s learning objectives are to help students grasp data parallelism, distinguish between load-balanced and load-imbalanced workloads, understand how the slowest processor bounds execution time, explain Amdahl’s law, recognize communication overhead, and compare theoretical vs. observed speedups. For a more detailed description of the activity, please refer to Chapter 1, Section 1.3.2.

Implementation Details

The material used for implementing this activity can be found at the following GitHub link: https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter4-USI/Unplugged_Activities/PENNY_SORTING. This activity is a self-contained instructional module structured across four main phases:

- a single-processor baseline (one student sorts, one times),
- a balanced two-processor parallel execution (two students split pennies equally, one times),

- a highly imbalanced five-processor parallel scenario (four students get two pennies each, while the fifth handles the remaining majority), and
- a final parallel execution time bounded by the slowest student plus communication overhead, leading into a broader instructional arc that concludes with video resources, structured group reflection, and an optional feedback survey.

For detailed step-by-step instructions on how to implement this activity, please refer to Chapter 2, Section 1.3.2.

Table 4.10 summarizes the approximate timing of each component of the penny sorting session. The class opened with a brief framing of the activity and its connection to sorting, a topic students had encountered earlier in the course, followed by group formation and distribution of materials. The bulk of the period was devoted to the unplugged activity itself, in which student groups searched a pile of pennies for the oldest coin under three successive work-distribution phases: sequential, evenly parallelized, and deliberately unbalanced, and recorded the completion time of each phase. Students then consolidated their results in a group discussion before completing an engagement survey.

Table 4.10: USI CS1 Approximate Timing of Unplugged Penny Sorting Exercise

Activity	Duration
Introduction: framing the activity, connection to sorting covered in class, goal of finding the oldest penny	4 min
Activity overview and setup: group formation (groups of 5–6), role assignment, distribution of coin bags	7 min
Phase 1 (sequential): one student searches all 50 pennies, a second times the search, remaining students observe; record time, shuffle, and reset	16 min
Phase 2 (parallel, balanced): two students each search 25 pennies, a third records both times; parallel time taken as that of the slower student; shuffle and reset	13 min
Phase 3 (parallel, unbalanced): four students receive two pennies each and a fifth receives the remainder, with one student timing; parallel time taken as that of the slowest student; shuffle and reset	12 min
Group discussion: what aspects of parallel computing did we observe?	6 min
Whole-class debrief and questions: comparison of recorded times, speedup, and load balancing	9 min
Post-activity student engagement survey	5 min
Closing and takeaways	3 min
Total	75 min

Experience and Teaching Tips

The Unplugged Penny Sorting Exercise effectively fits within a standard 50- to 75-minute class, dedicating 20 to 25 minutes to the active phases and the remainder to discovery-based discussion. Pedagogically, the activity relies on firsthand experience to build intuition: the uniprocessor phase establishes a slow baseline, Phase 2 reveals tangible parallel speedup, and

Phase 3 introduces a stark load imbalance where underutilized students sit idle, offering a perfect entry point to discuss processor starvation. Real-world variance in student sorting and communication times should be embraced as a teaching tool to illustrate how overhead prevents real systems from reaching perfect theoretical speedups. To scale the activity, instructors can use teaching assistants to monitor group mechanics in larger classes or pivot to a front-of-room volunteer demonstration for massive lecture halls. Ultimately, the exercise is most impacting when treated as a discovery experience, allowing students to analyze their own timing data and intuitively derive parallel computing concepts.

Data and Analysis

The participating students completed an anonymous engagement survey administered through Google Forms that contained Likert-scale items that measure student engagement, each rated on a five-point scale from 1 (Strongly disagree) to 5 (Strongly agree). The student engagement survey was adapted from the *Assessing Student Perspective of Engagement in Class Tool (ASPECT)* [7]. Several survey items were modified to reference the PDC activity and its learning objectives, and two PDC-specific questions were added to evaluate outcomes unique to this instructional intervention. The survey retained the three primary ASPECT categories: *Value of the Group Activity*, *Personal Effort*, and *Instructor Contribution*. Figure 1.3 illustrates the modified version of the ASPECT survey that was used for measuring student engagement for this activity. A total of thirty three students responded. Engagement items were organized into four dimensions: behavioral (effort, focus, participation), emotional (enjoyment, interest), cognitive (understanding and sense-making), and social (peer contribution). Two categories of items were set aside from the core engagement analysis: a baseline item measuring prior PDC knowledge, and four items measuring instructor delivery (preparedness, effort, availability), which reflect teaching quality rather than student engagement.

Engagement was strongly positive across the board. The average of the engagement item means is approximately 4.15 on the 5-point scale, and most items reached agreement (a rating of 4 or 5) from more than 80% of respondents. Figure 4.4 presents a diverging stacked bar chart, where each bar is centered on the neutral response (3), so disagreement extends left and agreement extends right. Bars are centered on the neutral response; item-label color denotes engagement dimension. The percentage agreeing (ratings of 4 or 5) is printed at the right of each bar, and item labels are colored by engagement dimension. Emotional and social items cluster at the far right with little to no left-side disagreement, while the three peer-learning cognitive items (orange) carry visibly more neutral and disagreeing responses.

Overall, the Unplugged Penny Sorting Exercise generated high engagement, driven most strongly by enjoyment, interest, and positive peer dynamics. The baseline prior-knowledge item was low (mean 2.67), which is expected and appropriate as it indicates that students entered with limited PDC familiarity, so the activity was introducing genuinely new material. The main actionable signal is the softer response to peer-based sense-making. Students valued the activity’s effect on their own understanding more than they valued explaining to, or learning from peers. In the future iterations, this can be addressed by structuring the group discussion component more deliberately.

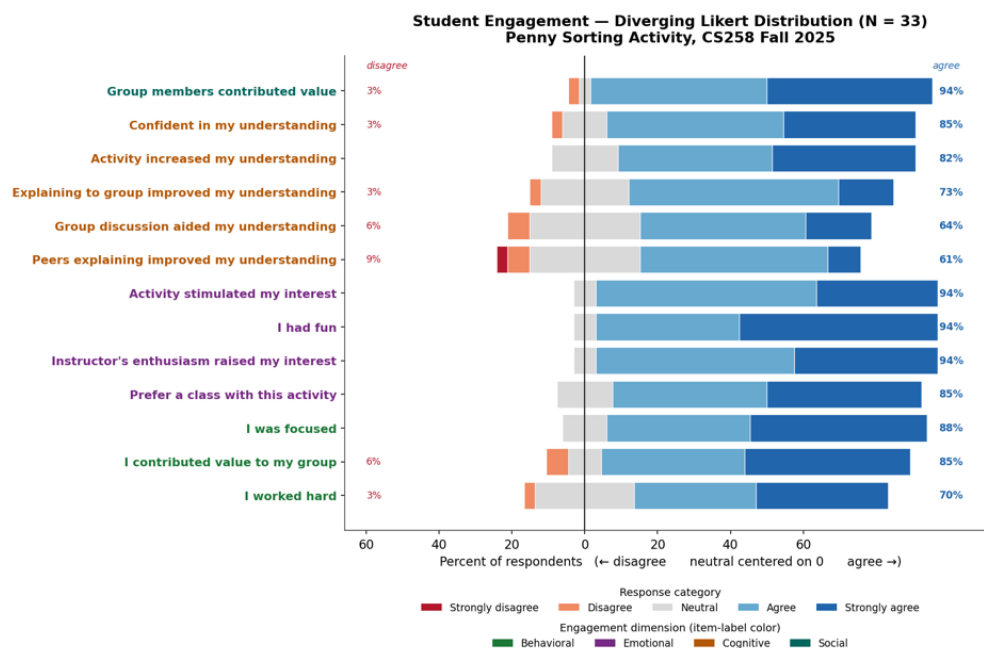


Figure 4.4: USI CS1 Diverging Likert Distribution of Engagement Items (n = 33)

4.4 New Plugged-in Activities

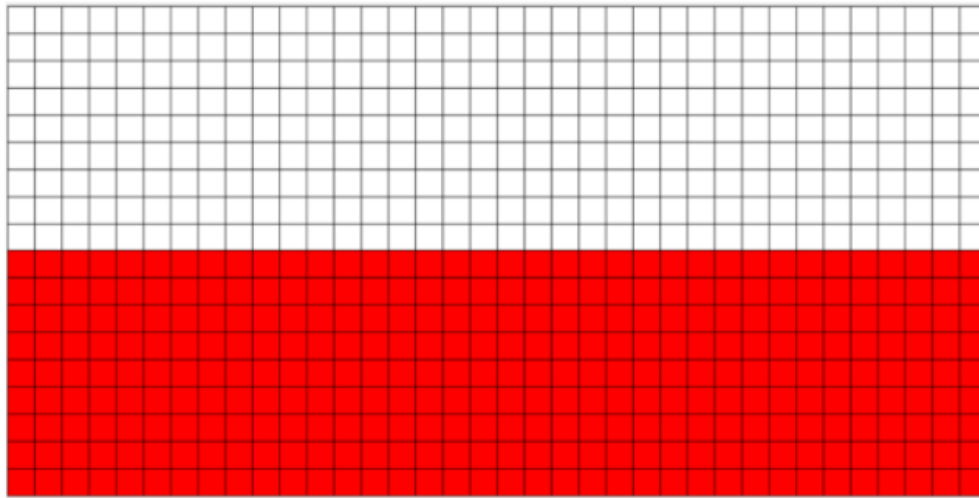
4.4.1 Flag Maker Plugged-In Activity

Problem Description, Prerequisites, and Learning Outcomes

The Flag Maker Plugged-In Activity is a programming assignment in which CS1 students design and implement algorithms to draw a series of international flags using a grid-based graphical environment. Although this activity is described as a PDC intervention for a CS1 course, it is inherently a sequential implementation of flag coloring. However, completing this activity before the parallel, Unplugged Flag Maker Activity prepared the students to better understand workload distribution when coloring a flag. Students are required to analyze each flag's geometric patterns, proportions, colors, and stripe layouts, then develop Java code using loops, conditional logic, variables, and coordinate-based grid manipulation to accurately generate scalable flag designs of varying sizes. The assignment includes flags such as Poland, Ukraine, Angola, Bolivia, Ireland, Rwanda, India, and a rainbow flag, with specific mathematical constraints related to height, width, and color distribution. An example of a gridded flag layout and corresponding code sample is illustrated in Figure 4.5.

As a prerequisite, to successfully complete the activity, students should possess foundational programming knowledge including variables, arithmetic expressions, nested loops, conditional statements, method structure, object-oriented concepts, and basic use of graphical grid APIs. They must also be able to reason about decomposition, coordinate systems, and proportional calculations to translate visual patterns into algorithmic solutions. The project assumes familiarity with Java development tools such as VS Code and the provided project framework.

Example when grid is size 18



Code sample:

```
int w = grid.getNumCols();
int h = grid.getNumRows();
for (int c=0; c < w; c++) {
    for (int r=h/2; r < h; r++) {
        grid.setColor(r, c, Color.RED);
    }
}
```

Figure 4.5: USI CS1 Example Flag Represented as a Colored Grid and Corresponding Code

Upon successful completion of this activity, students will be able to:

- Identify fundamental programming constructs including variables, loops, conditionals, methods, and grid-based graphical operations used in Java programs.
- Explain how national flag designs can be decomposed into geometric patterns, color regions, and proportional relationships that can be represented algorithmically.
- Implement nested loops, arithmetic calculations, and conditional logic to generate scalable flag designs that conform to specified dimensions and color arrangements.
- Test and verify that flag-generation algorithms produce correct output for multiple grid sizes rather than only the provided examples, identifying and correcting logical errors as needed.

Implementation Details

The material used for implementing this activity can be found at the following GitHub link. This activity was implemented as an individual programming assignment and served as a follow-up application of concepts learned in the course module on loops and iterative problem solving. Students were provided with a Java-based project framework and asked to complete a series of functions that generated multiple international flag designs using a grid-based graphical environment. The assignment required students to use nested loops, arithmetic calculations, variables, and conditional logic to color specific regions of a grid according to the proportions and patterns of each flag. Students were expected to create scalable solutions that worked correctly for different grid sizes rather than hard-coded dimensions. The assignment included progressively complex flag designs that required students to reason about proportions, coordinate systems, and pattern generation using loops.

Table 4.11 summarizes the timing of the Flag Maker Plugged-In Activity. The project was introduced during the first ten minutes of a regular 75-minute class session, in which the instructor framed the task, demonstrated the provided Poland flag example at several grid sizes, walked through the supporting code, and covered setup and submission logistics. Students then completed the project in pairs over the following week, drawing seven additional flags of increasing structural complexity. Because the graphical interface and one worked example were supplied, student effort was directed toward decomposing each flag into row and column ranges rather than toward interface programming.

Table 4.11: USI CS1 Approximate Timing of the Flag Maker Plugged-In Activity

Activity	Duration
Introduction to the project	2 min
Live demonstration of the provided Poland flag example, run at several grid sizes	3 min
Walkthrough of the provided code: the pre-coded GUI, <code>FlagMaker.java</code> , and the nested-loop sample that renders Poland	2 min
Project setup instructions: workspace file, selection of the platform-specific required Java extensions, and common pitfalls	2 min
Assignment logistics: pair formation, one-week deadline, grading rubric, and submission procedure	1 min
Regular class lecture: iteration (loops) in programming	65 min
Total (class period)	75 min
Out-of-class paired programming: environment setup, implementation of the remaining seven flags, testing at multiple grid sizes, and submission	1 week

This activity was completed before students were introduced to the Unplugged Flag Maker Activity. As a result, students initially approached the problem using sequential programming techniques and loop-based algorithms. Their experience developing solutions for drawing flags in code provided a foundation for later discussions about task decomposition, workload distribution, and parallel execution in the unplugged activity, where similar flag-design problems were explored from a parallel computing perspective. This sequencing

allowed students to first master the underlying algorithmic concepts before extending their understanding to parallel problem-solving approaches.

Experience and Teaching Tips

The assignment required students to analyze the mathematical structure of each flag, decompose complex visual designs into simpler programming tasks, and develop scalable solutions that worked for multiple grid sizes rather than fixed examples. Through hands-on coding, testing, and debugging, students gained practical experience applying fundamental programming concepts to solve visually engaging problems, while also strengthening computational thinking, problem decomposition, and collaborative software development skills. Allowing the students to work on the activity as a paired programming assignment improved the interest levels and enhanced student learning.

Data and Analysis

No data was collected in the form of surveys or pre/post test. The learning was assessed by grading the programming assignment that the students submitted. A total of 37 out of 39 students completed the assignment. Each student successfully completed at least 8 out of the 10 flags provided in the assignment. Overall, the assignment seemed to improve the understanding of the concepts of loops and iterative problem solving for students. This assignment also helped the students when they worked on the unplugged activity as they could easily translate their coding experience to the on-paper experience.

4.4.2 Parallel Penny Sort Plugged-In Activity

Problem Description, Prerequisites, and Learning Outcomes

In this activity, CS1 students were tasked with developing a Java application that simulates sorting a collection of pennies based on their minted years. Students generated an array of 50 random penny years, sorted the data using both a sequential sorting method and Java's parallel sorting functionality, measured the execution times of each approach, and identified the oldest penny in the collection. The project also encouraged students to analyze and compare sorting performance and, for extra credit, implement a binary search to locate a user-specified penny year within the sorted array.

As a prerequisite, before completing this activity, students were expected to have foundational knowledge of Java programming, including variables, control structures (loops and conditionals), methods, arrays, and basic algorithmic problem solving. Students also needed familiarity with generating random values, traversing arrays, and writing modular programs. Understanding of sorting concepts and basic program testing techniques was beneficial for successfully implementing and evaluating the required solutions. The activity served as a culminating project following instruction on arrays and introductory algorithmic techniques.

Regarding learning outcomes, upon completing this activity, students were expected to:

- Store and manipulate integer data using Java arrays.

- Implement and evaluate sorting algorithms using both sequential and parallel approaches.
- Generate and process randomized datasets for program testing and validation.
- Develop algorithms to locate the minimum value within a dataset and identify the oldest penny.
- Measure, compare, and analyze algorithm performance using execution-time data collected over multiple runs.
- Apply software engineering practices such as modular design, method decomposition, documentation, and testing to create readable and maintainable code.
- (Optional) Implement binary search on a sorted array and interpret search results.

Implementation Details

The material used for implementing this activity can be found at the following GitHub link. This activity was implemented as a team-based programming assignment following the Unplugged Penny Sorting Exercise. Students designed, developed, tested, and analyzed a Java application that worked with arrays of penny mint years. Students generated an array of exactly 50 randomly produced years representing pennies, displayed the unsorted data, and then applied two different sorting approaches: a sequential sorting method (such as a manually implemented bubble, insertion, or selection sort, or `Arrays.sort`) and Java's built-in `Arrays.parallelSort` method. Students measured and compared the execution times of both sorting techniques across multiple runs and reported average, minimum, and maximum execution times. After sorting the data, students implemented a single-pass algorithm to identify the oldest penny (minimum year) in the collection. The project emphasized modular programming, testing, code documentation, and performance analysis. As an optional enhancement, students could implement a binary search algorithm to locate a user-specified year within the sorted array and report whether the value was found.

Here are the instructions that were provided to the students in the assignment documentation:

1. Generate an array of 50 integer values representing penny mint years using a random number generator.
2. Use Java arrays (`int[]`) as the primary data structure and avoid using `ArrayList` or streams for the core logic.
3. Display the original unsorted array of penny years.
4. Implement a sequential sorting solution using a manual sorting algorithm (e.g., selection, insertion, or bubble sort) or `Arrays.sort`.
5. Implement a parallel sorting solution using `Arrays.parallelSort`.
6. Measure and record the execution time for both sorting approaches.

7. Execute each sorting approach multiple times and calculate summary statistics such as average, minimum, and maximum run times.
8. Sort the array in ascending order so that the oldest penny appears first.
9. Print the sorted array, the oldest year found, and a summary of the timing analysis.
10. (Optional Extra Credit) Implement a binary search method that allows a user to search for a specific penny year and report its location or indicate that it was not found.
11. Submit a zipped folder containing source code, a README file, presentation materials, and analysis of the sorting algorithms.
12. Demonstrate the completed project and present findings during the final project presentation.

Table 4.12 summarizes the timing of the Parallel Penny Sort Plugged-In Activity. The project was introduced during a regular 75-minute class session that opened with team formation, followed by a description of the problem statement and requirements and an extended Q&A period. Students spent the remainder of the session working within their teams to decompose the project into distinct tasks, assign ownership, and raise final questions with the instructor present. Teams then completed the implementation over the following week, timing a sequential and a parallel sort of the same data and analyzing the results, with a code demonstration and presentation due on the final exam day.

Table 4.12: USI CS1 Approximate Timing of the Penny Sort Plugged-In Activity

Activity	Duration
Team assignment: formation of project teams and distribution of the project instructions	5 min
Project description: problem statement, learning objectives, sequential and parallel sorting requirements, timing and analysis expectations, deliverables, rubric, and team-work guidelines	10 min
Question-and-answer session on project requirements and expectations	10 min
In-class team work: task breakdown and assignment among team members, setup of the project management board, planning of the timing analysis, and follow-up questions to the instructor	50 min
Total (class period)	75 min
Out-of-class team programming: implementation of the sequential and parallel sorts, repeated timing runs, analysis, documentation, and preparation of the demonstration and presentation	1 week

Experience and Teaching Tips

A primary challenge for students was implementing the parallel sorting method and verifying that it accurately ordered the data from oldest to newest. To achieve a correct

implementation, students had to research parallel sorting techniques, grasp their underlying mechanics, and understand their overall functionality. Working in teams greatly facilitated this research process, as group discussions helped most students successfully conceptualize parallel sorting.

For the extra credit component, students had to recognize the prerequisite that a binary search only functions on sorted data, and then correctly implement logic to return either the matching year's index or a "not found" result. This added complexity deterred many students from attempting the optional section. In future iterations, however, this search component could be integrated into the activity as a core requirement.

Data and Analysis

This activity was assessed by grading the programming assignments submitted by student teams and evaluating their performance during an in-class presentation. During their presentations, students explained their research into parallel sorting algorithms and connected their coding work back to the unplugged penny sorting activity, noting that the hands-on exercise significantly helped them program the parallel concepts. To improve this assessment in future iterations, the evaluation could be enhanced by introducing a structured grading rubric for the presentation, incorporating a peer-evaluation component to track individual contributions within teams, and utilizing a post-activity reflection survey to more objectively capture student learning gains.

4.4.3 Greenfoot Plugged-In Demo (time permitting/optional for CS1)

Problem Description, Prerequisites, and Learning Outcomes

As undergraduate computer science students transition from CS1 to CS2, they often face a steep learning curve when shifting from purely sequential, console-based programming to complex, non-linear execution models. Introducing advanced PDC paradigms like event-driven programming concepts too abruptly can overwhelm learners. To bridge this pedagogical gap, a specialized module was designed for the conclusion of the Fall 2025 CS1 course. The goal was to provide a visual, intuitive, and low-stakes sneak peek into CS2 fundamentals, demonstrating how asynchronous events interrupt sequential execution and map to real-world software interactions.

Students entering this module were required to have a foundational understanding of core CS1 concepts, including basic Java syntax, control structures (loops and conditionals), variables, and rudimentary object-oriented principles such as classes and methods.

The primary learning outcomes for this introductory module were for students to be able to:

- Identify the core differences between sequential execution and event-driven execution models.
- Describe how a graphical environment listens for, captures, and responds to asynchronous user inputs (e.g., keyboard and mouse events).

- Recognize the conceptual alignment between event handling and broader concurrent or parallel computing paradigms, laying a conceptual foundation for future CS2 coursework.

Implementation Details

A number of Greenfoot activities can be found on their website at: <https://www.greenfoot.org/home>. The module was implemented using Greenfoot [3], an educational visual Java development environment that combines an interactive object-oriented world with standard Java code. Toward the end of the Fall 2025 semester, a live interactive demonstration was conducted. The demo utilized a simple 2D simulation where “Actor” objects reacted in real-time to user interrupts. By examining the environment’s underlying execution loop (the `act()` method) alongside keyboard listeners (such as `Greenfoot.isKeyDown()`), the mechanics of event loops and event handlers were explicitly deconstructed. This visual feedback allowed students to see how real-time user inputs break the traditional top-down flow of execution, introducing them to the asynchronous nature of graphical user interfaces and concurrent systems.

The instructional delivery relied on a blend of synchronous demonstration and supportive multimedia resources, including:

- **Interactive Lecture and Live Coding:** A guided session introducing the Greenfoot interface, contrasting a traditional console-based Java program with an event-driven graphical environment.
- **Curated Video Tutorials:** Short, targeted video walkthrough from the Greenfoot repository were provided to students. These videos reinforced how the platform manages spatial coordination, actor interaction, and background event listeners, allowing students to revisit the mechanics at their own pace.

Table 4.13 summarizes the approximate timing of the Greenfoot session, conducted at the end of the Fall 2025 semester.

Experience and Teaching Tips

Utilizing a highly visual, gamified environment successfully lowered the barrier to entry for a complex topic. Students demonstrated high engagement, noting that seeing an immediate graphical response to a keyboard event made the abstract concept of listeners and interrupts concrete.

Lessons Learned and Teaching Tips for Future Iterations:

- **Emphasize the Execution Loop:** Spend sufficient time explaining that a background engine is continuously polling for events. This prevents the common misconception that the code written inside an actor is running in isolation.
- **Bridge Explicitly to CS2:** Explicitly use the vocabulary of CS2 (such as interfaces, inheritance, and concurrency) during the demo so that the terminology feels familiar when students encounter it formally in the subsequent semester.

Table 4.13: USI CS1 Approximate Timing of the Greenfoot Plugged-In Demo

Activity	Duration
Introduction and framing: motivation for event-driven execution, contrast between a traditional console-based Java program and an interactive graphical environment	5 min
Greenfoot demonstration and live coding: tour of the Greenfoot interface and a 2D simulation showing mechanics of event loops and event handlers	20 min
Student hands-on exploration: students modify actor behavior and experiment with real-time keyboard input, observing how user events break top-down flow of execution; curated video tutorials from the Greenfoot repository available for self-paced review, with the instructor present for questions	45 min
Closing discussion and questions: the asynchronous nature of graphical user interfaces, connection to concurrent systems, and semester wrap-up	5 min
Total	75 min

Data and Analysis

Because this module was executed strictly as an introductory demonstration and an active learning preview, no formal quantitative data or assessment metrics were collected during the Fall 2025 semester. Formal data collection and empirical analysis are planned for a follow-up semester in the CS2 course, where students will be writing their own event-driven programs and exploring explicit PDC components. Assessment methodologies for the CS2 implementation will include code analysis rubrics and qualitative surveys regarding event based programming concepts.

4.5 Course-wide Pre and Post Course Surveys

To evaluate the impact of integrating PDC concepts into the introductory computer science curriculum, a comparative data collection study was conducted across two consecutive fall semesters. In Fall 2024, baseline data was established under a traditional curriculum structure without any explicit PDC interventions. To assess student growth and conceptual shifts, identical pre- and post-surveys were administered during the first and final weeks of the semester, respectively. The survey content is described in more detail in Chapter 1, Section 1.5. In Fall 2025, the same pre- and post-survey methodology was employed to measure the efficacy of a targeted pedagogical intervention consisting of three distinct PDC modules. As described in the sections above, this intervention integrated two “unplugged” and two “plugged-in” activities, collectively exposing students to core concurrent computing concepts including data parallelism, parallel speedup, task scheduling, and load balancing, as well as a foundational demonstration of event-driven programming using the Greenfoot environment. A key structural difference between the two semesters is the primary programming language: C# was used in Fall 2024, whereas Java was adopted for Fall 2025. Consequently, the subsequent data analysis acknowledges that this shift in language may act as a confounding factor. Nevertheless, the overall results strongly indicate that the PDC intervention effectively improved students’ self-reported understanding of parallel and distributed computing

concepts.

4.5.1 Results and Analysis: Fall 2024 Benchmark Survey Data

Of 26 pre-course and 18 post-course unique respondents, 13 students completed both surveys and form the paired sample. The matched baseline cohort is small (13), split across freshman/sophomore/junior standing. Nearly all expected an A. An analysis of the pre/post survey results collected during Fall 2024 (baseline semester) is summarized in Table 4.14. Figure 4.6 shows the shifts in the Likert scale (7-point scale) values for the questions on the pre- and post-surveys. All 14 matched students answered the three free-text items.

Table 4.14: USI CS1 Pre/Post Survey Results from Benchmark Fall 2024

Category	<i>n</i>	Mean (pre)	Mean (post)	Mean Diff	<i>W</i>	<i>p</i> -value	Sig?	Cliff δ	Interpretation
Computing Background									
experience-computer	13	5.77	6.15	0.38	3	0.188	No	-0.17	small
experience-programming	13	4.15	4.54	0.38	6	0.250	No	-0.15	small
experience-peers	13	3.77	4.54	0.77	3.5	0.027	Yes	-0.26	small
Programming Concepts									
Data/Variable Types	13	4.15	5.23	1.08	7	0.021	Yes	-0.25	small
Arithmetic Expr. and Operators	13	4.31	5.23	0.92	7	0.043	Yes	-0.19	small
Branching	13	4.38	5.08	0.69	6	0.133	No	-0.14	negligible
Loops	13	4.23	4.77	0.54	14	0.363	No	-0.08	negligible
Calling functions/methods	13	4.31	4.92	0.62	6	0.125	No	-0.14	negligible
Arrays	13	3.46	4.69	1.23	7	0.043	Yes	-0.30	small
Writing static functions	13	3.38	4.85	1.46	3	0.006	Yes	-0.44	medium
Writing methods	13	3.92	5.00	1.08	7	0.045	Yes	-0.26	small
Writing classes	13	3.46	5.00	1.54	3	0.012	Yes	-0.41	medium
Creating objects	13	3.54	4.85	1.31	5	0.047	Yes	-0.33	small
PDC Concepts									
parallel	13	2.08	3.62	1.54	3.5	0.003	Yes	-0.59	large
distributed	13	2.00	3.08	1.08	3	0.023	Yes	-0.44	medium
event	13	3.08	3.77	0.69	16.5	0.169	No	-0.23	small
Computing Attitudes									
computing interest	13	6.15	6.31	0.15	6	0.531	No	-0.24	small
PDC interest	13	5.77	5.85	0.08	16.5	0.961	No	-0.03	negligible
PDC importance	13	5.85	6.15	0.31	5.5	0.406	No	-0.22	small
computing understanding	13	4.77	5.31	0.54	3.5	0.109	No	-0.17	small
PDC understanding	13	3.46	4.08	0.62	13	0.289	No	-0.23	small

Pre-course aspirations: The clearest theme is that goals were framed around C#:

- student quotes on learning C# as a language (most common): “learning the C# syntax,” “fully learning C#,” “learn the syntax of C# as I’ve never used the language,” “write high level programs in C#.” One student tied it to C# based game engines.
- student quotes on general fundamentals: “different ways to think about programming,” making code “more efficient,” a “platform to begin enhancing my programming abilities.”

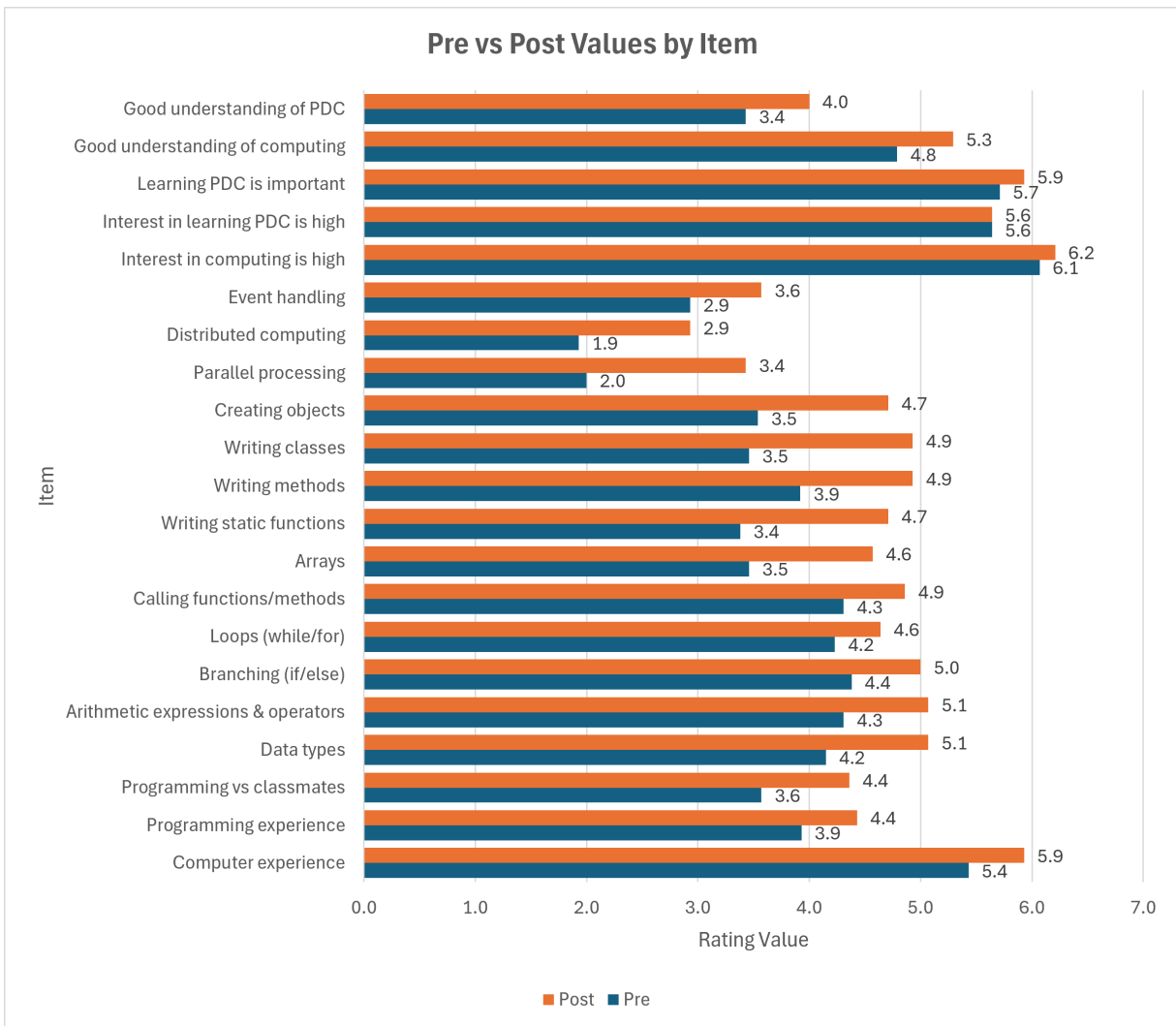


Figure 4.6: USI CS1 Fall 2024 Pre to Post Shift on Every Survey Item

What helped most: students credited hands-on, example-driven work, with peer collaboration and instructor quality:

- Quotes on code examples worked through in class (dominant): “coding examples in class were mostly how I learned C#,” “examples of what the code should look like and why.”
- quotes on hands-on practice and homework: “programming assignments in lecture and in-class practices,” “working on segments of the projects in class.”
- quotes on peer collaboration and a caring instructor: “working with other people to problem solve,” “getting help from others,” “a professor that cares... and passionate.”

Suggested adjustments: change requests centered on materials and assessment more than teaching style:

- quotes on textbook and materials: “the book could be optional,” “a better textbook,” “a more interactive way... to learn C#.”
- quotes on exams and testing: “make exam questions less complex,” more clarity on coverage, and one arguing tests “don’t actually show you if you know how to do it.”
- quotes from satisfied students: several saw little to change, one noted the class was “very easy” given their prior experience.

4.5.2 Results and Analysis: Fall 2025 PDC Intervention Survey Data

Of 41 pre-course and 27 post-course respondents, 25 students completed both surveys (2 students submitted partially complete surveys) and form the paired sample analyzed here. Ordinal responses were mapped to 1–7 numeric scales for effect-size estimation. The result demonstrate broad, large, statistically robust gains in programming and PDC experience, corroborated by the open-ended comments. The sample has predominantly Computer Science majors (26 of 27), largely traditional age (23 of 27 aged 18–22), and skewing early-undergraduate, though juniors and sophomores are well represented. Most students entered expecting an A or B. An analysis of the pre/post survey results collected during Fall 2025 (PDC intervention semester) is summarized in Table 4.15. Figure 4.7 shows the shifts in the Likert scale (7-point scale) values for the questions on the pre- and post-surveys.

On nearly every construct the course targets, students moved roughly two full points on a seven-point scale between week 1 and the final week. The biggest jumps cluster in the object-oriented core : writing methods (+2.7), creating objects (+2.7), writing classes (+2.6), and in the PDC intervention (parallel processing +2.4, distributed computing +1.9). Four items did not shift significantly: computer experience, general interest in computing, interest in learning PDC, and (borderline) belief that PDC is important. All four were already high at pre-survey (means of 5.9, 6.4, 5.8, 5.7 on the 7-pt scale). Thus this can be regarded as a ceiling effect rather than a failure of the course.

Three text response items were coded thematically: pre-course aspirations (27 responses), and post-course “what helped most” (26) and “what to change” (20).

1. Pre-course aspirations: students entered with grounded goals rather than specialized ones. Four themes recurred:
 - Building basic fluency and confidence (most common): “be able to program myself,” to grasp “basic coding fundamentals,” or to get a “base start.”
 - Java & object-oriented programming: understanding Java, “object based programming,” better style and “optimization of code.”
 - Career & real-world application: “my career path,” “the real world,” even “the enterprise level.”
 - Conceptual understanding of how computers work: a reflective minority wanted the “why,” not just the “how.”

Table 4.15: USI CS1 Pre/Post Survey Results from PDC Intervention Fall 2025

Category	<i>n</i>	Mean (pre)	Mean (post)	Mean Diff	<i>W</i>	<i>p</i> -value	Sig?	Cliff δ	Interpretation
Computing Background									
experience-computer	25	6.04	6.00	−0.04	22.5	1.000	No	0.03	negligible
experience-programming	25	3.48	4.68	1.20	6	0.000	Yes	−0.50	large
experience-peers	25	3.24	4.40	1.16	10	0.001	Yes	−0.43	medium
Programming Background									
Data/Variable Types	25	3.04	5.28	2.24	7.5	0.000	Yes	−0.61	large
Arithmetic Expr. and Operators	25	3.08	5.08	2.00	10	0.000	Yes	−0.60	large
Branching	25	3.40	5.32	1.92	8.5	0.000	Yes	−0.62	large
Loops	25	3.48	5.32	1.84	17.5	0.000	Yes	−0.58	large
Calling functions/methods	25	2.80	5.16	2.36	2.5	0.000	Yes	−0.62	large
Arrays	25	2.12	4.52	2.40	3	0.000	Yes	−0.66	large
Writing static functions	25	2.20	4.64	2.44	3.5	0.000	Yes	−0.72	large
Writing methods	25	2.16	4.88	2.72	0	0.000	Yes	−0.72	large
Writing classes	25	2.28	4.92	2.64	0	0.000	Yes	−0.70	large
Creating objects	25	2.36	5.08	2.72	0	0.000	Yes	−0.80	large
PDC Background									
parallel	25	1.76	4.12	2.36	0	0.000	Yes	−0.75	large
distributed	25	2.04	3.84	1.80	15.5	0.000	Yes	−0.56	large
event	25	3.32	4.32	1.00	20.5	0.013	Yes	−0.31	small
Computing Attitudes									
computing interest	25	6.36	6.24	−0.12	25	0.448	No	0.06	negligible
PDC interest	25	5.76	5.52	−0.24	44	0.341	No	0.16	small
PDC importance	25	5.68	6.12	0.44	32	0.052	No	−0.26	small
computing understanding	25	4.36	5.56	1.20	12	0.000	Yes	−0.46	medium
PDC understanding	25	3.24	5.40	2.16	4.5	0.000	Yes	−0.77	large

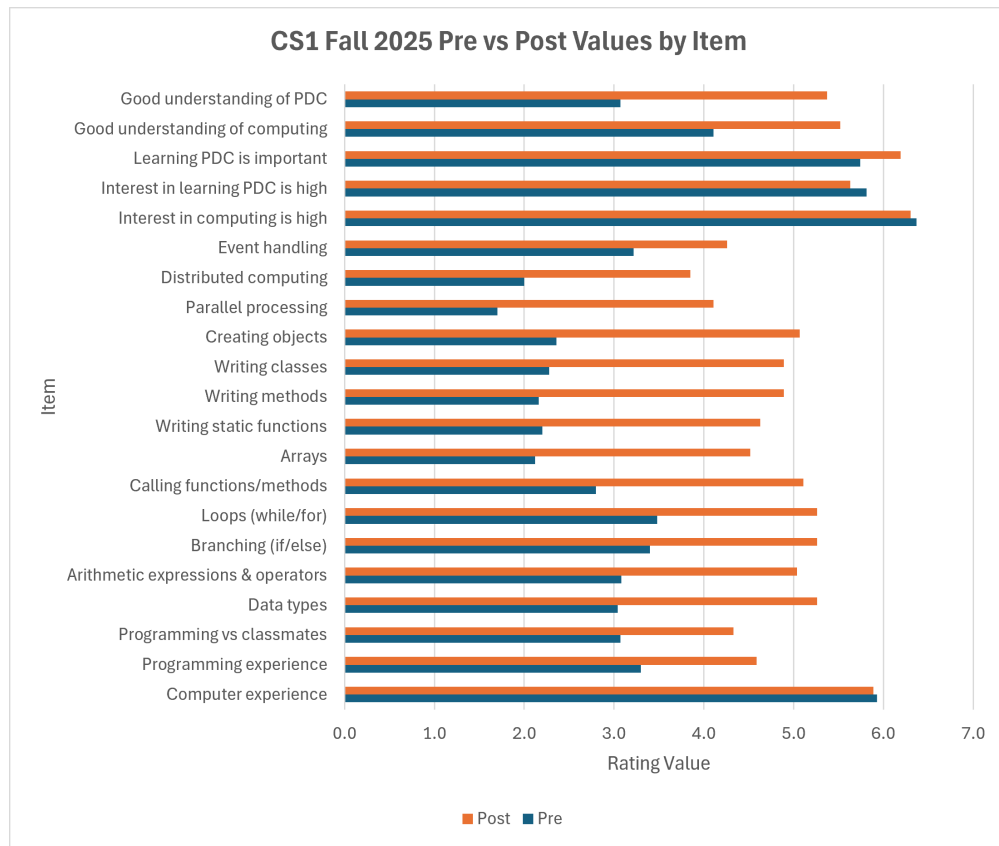


Figure 4.7: USI CS1 Fall 2025 Pre to Post Shift on Every Survey Item

2. What helped most: a near-consensus for active learning

- Hands-on, in-class coding & worked examples (20 of 26) — “in-class practices,” “writing code in general,” demos worked through live, and “seeing what the code is supposed to look like after I attempt my version.”
- Projects & from-scratch assignments: the midterm/final projects and “writing an entire code from scratch” were singled out as the deepest learning.
- The unplugged PDC activities: three students spontaneously praised the “unplugged activities” and the “penny sort” demonstration. This is direct evidence the successful PDC intervention.

3. Suggested adjustments: the change requests were constructive and mostly asked for more of what already worked:

- More active coding, fewer slides (dominant): “more hands on doing the code ourselves than powerpoints,”
- More projects & recurring mini-labs: including “mini labs every two-three weeks” to refresh earlier modules.
- Accountability: some wanted in-class participation graded

- Pacing & challenge: a couple felt the pace fast (“too much too quickly”). Others wanted more challenge. Several were fully satisfied.

4.5.3 Did the PDC Intervention Improve Learning?

To evaluate the impact of the intervention, a difference-in-differences (DiD) approach was utilized to compare the evaluation data between the Fall 2024 and Fall 2025 semesters. For each metric, the average learning gain (defined as post-survey score – pre-survey score) was computed for each cohort. The final treatment effect was estimated by subtracting the 2024 baseline gain from the 2025 intervention gain, isolating the marginal learning increase directly attributable to the PDC activities. A comparison of individual items between the Fall 2024 baseline and Fall 2025 intervention semesters is presented in Figure 4.8. On the composite measure of PDC concepts and self-rated understanding, the intervention cohort outperformed the baseline cohort in total points gained. Furthermore, the share of improving students increased from 79% to 93% in Fall 2025. These results suggest that the intervention nearly doubled self-reported PDC learning gains from a comparable baseline.

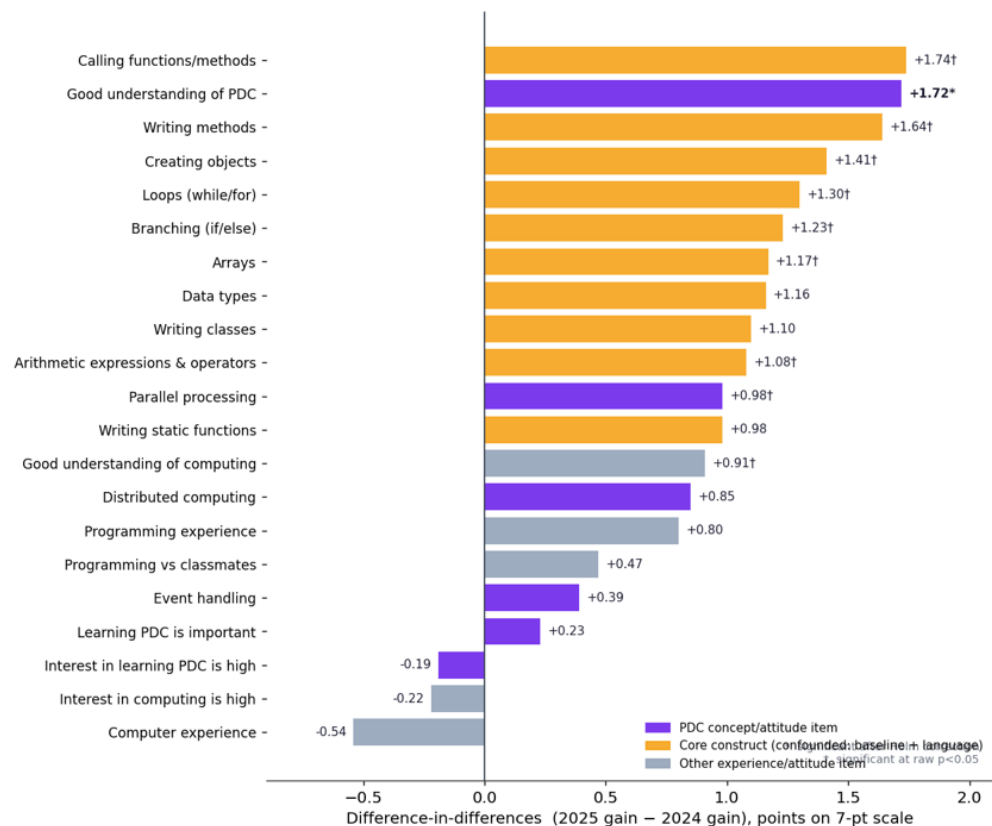


Figure 4.8: USI CS1 Improvement of Each Item from Benchmark Fall 2024 to PDC Intervention Fall 2025

Figure 4.9 shows the specific PDC concept gains between the baseline cohort and the intervention cohort. Parallel processing and distributed computing experience each roughly doubled and are significant at the raw level. Event handling shows almost no difference

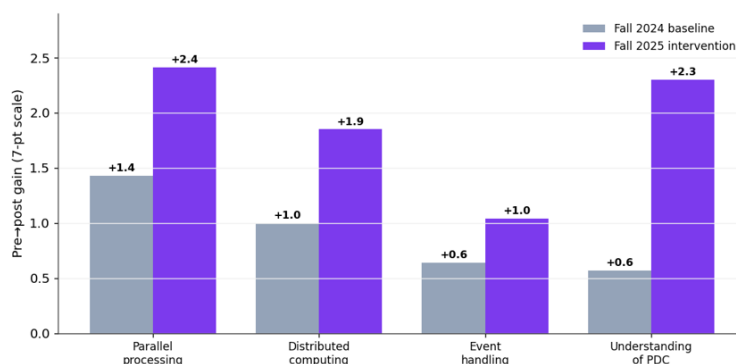


Figure 4.9: USI CS1 Pre to Post Gains on Four PDC Measures: Parallel Processing, Distributed Computing, and PDC Understanding

and is consistent with it being integrated as a demo of the concepts the students will learn in CS2. One thing to note from these results is that concepts such as writing methods, calling functions, creating objects and others show large DiD values, but the 2024 cohort started higher on these and the teaching language differed between the cohorts (C# to Java). Overall, using Fall 2024 as a baseline, the evidence indicates that the PDC intervention improved students' self-reported learning of parallel and distributed computing concepts. From starting points that were statistically identical across cohorts, the intervention year produced roughly double the gain on PDC concepts. An analysis of the combined pre/post survey results collected during Fall 2024 (baseline semester) and Fall 2025 (PDC intervention semester) are summarized in Table 4.16.

4.6 Other Activities and Ideas

4.6.1 More Processors are Not Always the Best

The core problem addressed by this module is demonstrating how adding more hardware execution units (processors) does not inherently lead to a linear decrease in runtime. Instead, performance scaling is heavily constrained by dependencies, multi-tasking overhead, coordination delays, and structural imbalances. This active learning module is explicitly designed for an early undergraduate environment. Basic exposure to the concept of how a CPU executes a sequence of instructions sequentially. No prior parallel programming experience or specialized hardware access is required. The material used for implementing this activity in class can be found at the following Github link.

Learning Outcomes: By completing this unplugged activity, students should be able to:

- Differentiate between uniprocessor sequential execution, uniprocessor multitasking, and true multi-processor parallel execution.
- Explain how communication, coordination, and synchronization overhead can impede parallel efficiency.

Table 4.16: USI CS1 Combined Pre/Post Survey Results from Fall 2024 and Fall 2025

Category	<i>n</i>	Mean (pre)	Mean (post)	Mean Diff	<i>W</i>	<i>p</i> -value	Sig?	Cliff δ	Interpretation
Computing Background									
experience-computer	38	5.95	6.05	0.11	43	0.296	No	−0.03	negligible
experience-programming	38	3.71	4.63	0.92	25.5	0.000	Yes	−0.36	medium
experience-peers	38	3.42	4.45	1.03	24	0.000	Yes	−0.38	medium
Programming Concepts									
Data/Variable Types	38	3.42	5.26	1.84	27	0.000	Yes	−0.48	large
Arithmetic Expr. and Operators	38	3.50	5.13	1.63	29	0.000	Yes	−0.44	medium
Branching	38	3.74	5.24	1.50	24	0.000	Yes	−0.41	medium
Loops	38	3.74	5.13	1.39	57	0.000	Yes	−0.39	medium
Calling functions/methods	38	3.32	5.08	1.76	15	0.000	Yes	−0.46	medium
Arrays	38	2.58	4.58	2.00	18	0.000	Yes	−0.54	large
Writing static functions	38	2.61	4.71	2.11	12	0.000	Yes	−0.62	large
Writing methods	38	2.76	4.92	2.16	11	0.000	Yes	−0.56	large
Writing classes	38	2.68	4.95	2.26	5.5	0.000	Yes	−0.60	large
Creating objects	38	2.76	5.00	2.24	11	0.000	Yes	−0.64	large
PDC Concepts									
parallel	38	1.87	3.95	2.08	5	0.000	Yes	−0.70	large
distributed	38	2.03	3.58	1.55	28	0.000	Yes	−0.52	large
event	38	3.24	4.13	0.89	70	0.004	Yes	−0.28	small
Computing Attitudes									
computing interest	38	6.29	6.26	−0.03	75	0.939	No	−0.04	negligible
PDC interest	38	5.76	5.63	−0.13	120	0.568	No	0.09	negligible
PDC importance	38	5.74	6.13	0.39	62	0.029	Yes	−0.24	small
computing understanding	38	4.50	5.47	0.97	27	0.000	Yes	−0.36	medium
PDC understanding	38	3.32	4.95	1.63	49.5	0.000	Yes	−0.60	large

- Identify execution dependencies that lead to idle processor time (e.g., waiting for data from an upstream process).
- Relate concrete physical execution blockages to abstract theoretical scaling laws, such as Amdahl’s Law limitations.

Implementation Guidelines: the session is conducted in a standard lecture room by organizing students into collaborative groups of 5–6 members, where participants rotate through roles as individual “processors” or dedicated “timers”. All execution processors are given their own local sheet of paper acting as an isolated local data buffer. The workload consists of two computational sub-tasks:

- Task 1: Writing out the string literal “more processors are not always the best”.
- Task 2: Assigning a sequential index value (0, 1, ... 32) directly beneath each individual character of the written sentence.

Figure 4.10 shows the activity phases and how PDC concepts are implemented in those phases.

Phase	Resource Configuration	Task Breakdown & Execution Dynamics	Performance Metric Logic
1. Uniprocessor Sequential	1 Active Worker, 1 Timer	Participant 1 executes Task 1 completely, followed sequentially by Task 2 on a single sheet.	Establishes the core baseline execution time.
2. Uniprocessor Multi-Tasking	1 Active Worker, 1 Timer	Participant 1 interleaves the workload character-by-character (writes a letter, then immediately writes its index).	Benchmarks the operational impact of context-switching/interleaving against the serial baseline.
3. Two-Processor Parallel	2 Active Workers, 1 Timer	Participant 1 writes the complete sentence (Task 1) while Participant 2 concurrently indexes it (Task 2) on their respective sheets from given start/end boundaries.	Total time is bound by the slower of the two concurrent workers.
4. Multi-Processor Parallel with Communication Overhead	4 Active Workers, 1 Timer	Data Partitioning: Workers 1 and 2 split the string writing workload (“more processors are” vs. “not always the best”). Workers 3 and 4 handle indexing. Worker 4 must explicitly wait to receive the final index boundary from Worker 3 before continuing.	Illustrates idle processor time and synchronization overhead. Total time is bound by the slowest processor.

Figure 4.10: USI CS1 Progressive Operational Activity Phases to Benchmark Execution Time

Lessons learned and teaching tips: ensure students strictly work on their own physical sheets of paper. If they attempt to share a single piece of paper simultaneously, it will inadvertently simulate hardware memory bus contention rather than the intended message-passing synchronization overhead. Guide students to explicitly connect their physical experience of waiting for a teammate’s indexing number to real-world parallel computing stalls and Amdahl’s Law restrictions.

A survey similar to the ASPECT based survey used for the Penny Sorting activity discussed above was collected after this activity as well. Survey results indicated that the activity attributed to high levels of student engagement and therefore an improved interest in PDC learning. A more detailed description of the activity and an in-depth analysis of the student engagement results collected after the activity implementation can be found in [6].

4.7 Conclusion

This chapter demonstrates that PDC concepts can be introduced meaningfully in a CS1 course without displacing core object-oriented objectives or consuming additional instructional days. The evidence assembled at USI across the Fall 2024 and Fall 2025 offerings of CS 258 supports the claim. A small number of well-placed “unplugged” activities, paired with targeted code implementations or “plugged-in” activities, and asynchronous visual demonstrations, produced statistically reliable gains on the PDC concepts those activities were designed to teach.

- Lessons Learned

- Pre- and post-test analyses revealed that certain PDC concepts failed to show expected learning gains because of specific activity design limitations. For instance, the lack of improvement regarding race conditions stems directly from how the tasks were structured. Both the Flag Maker and Penny Sorting activities involve distributing and decomposing independent workloads across processors without incorporating shared mutable state. Consequently, while students gained an excellent intuition for decomposition, speedup, and load balancing, they did not encounter or understand race conditions. This underscores a critical pedagogical lesson: concepts that are merely mentioned during an activity, rather than actively integrated, can be overshadowed and must be carefully reevaluated during future activity redesigns.
- Both the Fall 2024 and Fall 2025 cohorts suffered significant data attrition due to inconsistent, missing student identifiers. In Fall 2024, the sample size dropped from 27 pre-course respondents to 19 post-course respondents, ultimately yielding only 13 uniquely matched pairs after filtering out duplicate submissions and malformed email addresses. Similarly, the Fall 2025 cohort shrank from 32 pre-course to 27 post-course respondents, resulting in just 23 successfully matched pairs. To prevent this data loss in future iterations, surveys will be tied to a small grade incentive to boost completion rates, and by using standardized coded student IDs to stay anonymous and consistent.

- Teaching Tips for Instructors

- Before adopting an activity, ask which PDC concept its physical mechanics actually instantiate. For example, Flag Maker instantiates decomposition, speedup, and load imbalance, and it teaches those three superbly.
- For effective implementation, activities should use existing sessions rather than additional ones, and be positioned so that the accompanying code implementations are reinforced versions of existing assignments or projects.
- Pre-populate the identifier field, or use a roster-linked code for collecting student data. This single change would preserve a substantial fraction of the data lost pre- and post- survey collection.

- Inferences and Limitations

- the language of instruction changed from C# to Java between the two semesters. Since the design does not support a cross-semester comparison in any case, this is not a confound to be adjusted for a limit on generalization: the Fall 2025 result is a result about a Java-based CS1.
- the samples are small and subject to attrition. Twenty-three matched pairs in Fall 2025 and 14 in Fall 2024 leave little power for subgroup analysis.
- this chapter’s data do not speak to whether CS1 learning outcomes were preserved. The claim that PDC content did not displace core objectives rests on the structural fact that no instructional days were added and no core topics were removed.

4.7.1 Future Work

Several extensions follow directly from the limitations above. The most immediate is instrument redesign for establishing a stable, roster-linked identifier, and addition of a grade for completing the surveys. Beyond the CS1 boundary, the durability of these gains remains open. Tracking the Fall 2025 cohort into CS 358 (CS2) and later coursework would establish whether early intuitions persist and whether they accelerate later learning.

Bibliography

- [1] Steven A Bogaerts. Limited time and experience: Parallelism in cs1. In 2014 IEEE International Parallel & Distributed Processing Symposium Workshops, pages 1071–1078. IEEE, 2014.
- [2] Andrew T Kitchen, Nan C Schaller, and Paul T Tymann. Game playing as a technique for teaching parallel computing concepts. ACM SIGCSE Bulletin, 24(3):35–38, 1992.
- [3] Michael Kölling. The greenfoot programming environment. ACM Transactions on Computing Education (ToCE), 10(4), 2010.
- [4] Suzanne J. Matthews. Pdcunplugged: A free repository of unplugged parallel distributed computing activities. In 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pages 284–291, 2020.
- [5] Brandon Rodriguez, Stephen Kennicutt, Cyndi Rader, and Tracy Camp. Assessing computational thinking in cs unplugged activities. In Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education, SIGCSE ’17, page 501–506. Association for Computing Machinery, 2017.
- [6] Srishti Srivastava, Mary Smith, Amrita Ghimire, and Shaun Gao. Assessing the integration of parallel and distributed computing in early undergraduate computer science curriculum using unplugged activities. In 2019 IEEE/ACM Workshop on Education for High-Performance Computing (EduHPC), pages 17–24, 2019.
- [7] Benjamin L. Wiggins, Sarah L. Eddy, Lauren Wener-Fligner, Katherine Freisem, Daniel Z. Grunspan, Elli J. Theobald, Jessica Timbrook, and Alison J. Crowe. ASPECT: A Survey to Assess Student Perspective of Engagement in an Active-Learning Classroom. CBE—Life Sciences Education, 16(2):ar32, Summer 2017.

Appendix

Github Link to Instructional Material

The USI CS1 repository containing relevant PDC intervention material as described in this chapter can be found at: *USI-CS1-PDC-Github*.

Detailed Pre and Post Syllabus

The Parallel and Distributed Computing (PDC) intervention was implemented in CS 258, Introduction to Object-Oriented Programming, the CS1 course at the University of Southern Indiana. CS 258 is a three-credit, Java-based course covering fundamental object-oriented programming using Gaddis's Starting Out with Java: Control Structures through Objects (Pearson Revel Ed.). The pre-intervention syllabus followed a conventional CS1 structure, listing nine course learning objectives centered on introductory OOP concepts, object-oriented principles, design patterns, debugging, multithreaded applications, group collaboration, code review and documentation, and a final project, with no explicit treatment of parallel and distributed computing. The post-intervention syllabus preserved this structure but added a dedicated learning objective on parallel computing fundamentals, under which students explain sequential versus parallel computing, data parallelism, speedup, and parallel overhead, and apply existing parallel methods to optimize sequential code. The accompanying course event calendar integrates the PDC interventions directly into the existing nine module schedule (FlagMaker exercise within Module 4 (Loops and Files), a Penny Sorting activity as the Module 9 final project, and a Greenfoot event-driven programming demonstration). The pre- and post-course experience surveys were collected in the first and last weeks of the semester respectively. In this way, PDC concepts were woven into an otherwise standard CS1 sequence without displacing existing core course content.

Organization of Material

The chapter4-USI directory is organized as follows:

```
chapter4-USI/  
  Detailed_Pre_and_Post_Syllabus/  
  Evaluation_Data_and_Analysis/  
  Plugged-In_Activities/  
  Unplugged_Activities/  
  USI-CS1_CourseCalendar_Fall2025.pdf  
  readme.md
```

Detailed _Pre_and_Post_Syllabus

Purpose: This directory (available at *Detailed-Pre-Post-Syllabus*) contains detailed syllabus materials for pre- and post-course assessments.

Contents: Documentation and resources related to course prerequisites and learning outcomes established before and after course instruction.

Usage: Reference materials for understanding the syllabus structure and course requirements.

Evaluation_Data_and_Analysis

Purpose: This directory (available at *Evaluation-Data-and-Analysis*) houses data collection and analytical materials used for course evaluation.

Contents: Assessment data, evaluation metrics, and analysis documentation for measuring student learning outcomes and course effectiveness.

Usage: Source for course evaluation reports, student performance metrics, and instructional efficacy analysis.

Plugged-In_Activities

Purpose: This directory (available at *Plugged-In-Activities*) contains computer-based instructional activities that require digital resources and technology.

Contents: Programming exercises, computational thinking labs, hands-on coding activities, and digital learning modules designed to develop technical competencies.

Usage: Classroom instruction materials for technology-integrated lessons and practical programming exercises.

Unplugged_Activities

Purpose: This directory (available at *Unplugged-Activities*) contains non-digital instructional activities focused on foundational concepts.

Contents: Paper-based exercises, kinesthetic learning activities, collaborative group work, and conceptual activities that develop problem-solving skills without direct technology use.

Usage: Complementary classroom materials for reinforcing core concepts and engaging diverse learning styles.

USI-CS1_CourseCalendar_Fall2025.pdf

Purpose: Official course calendar (available at *USI-CS1-Course-Calendar-Fall2025*) for the Fall 2025 CS1 course at the University of Southern Indiana.

Contents: Academic schedule including lecture dates, assignment deadlines, examination dates, and important institutional dates.

File Type: PDF document

Usage: Primary scheduling reference for students and instructors.

readme.md

Purpose: Quick reference documentation for the USI module.

Contents: Overview, navigation guide, and key information about accessing and utilizing resources in this directory.

Usage: Entry point for users to understand the module's organization and purpose.

Survey and Other Anonymized Data and Analysis

On the GitHub repository, the directory *Evaluation-Data-and-Analysis*, contains the de-identified and anonymized data and the analytical materials used for the evaluation of the PDC interventions.